Referee Report: "Honest Code Benchmark" Preregistration, Research Plan, and Code

Materials reviewed: `preregistration.md`, `research-plan.md`, `task-specification.md`, `implementation-order.js` and `.json`, `generate-data.js`, the `data/` directory, the `apps/` directory (all 55 arm-archetype folders, both servers, the build script), and the `harness/` directory. **Source:** GitHub clone of `openhonest/honest-code-benchmark`, commit `0754d6a3ffdfb19f95bda2175f9747b291301e0e` (Oct. 29, 2026, 4:35 p.m. CT). The repository has two commits. All code, data, and the implementation order arrived together in the first commit (`198d930`, Oct. 29, 2026, 1:46 p.m. CT). **Target venue:** *Software: Practice and Experience*. **Review date:** Oct. 29, 2026. Read-only review. I did not install dependencies, run the servers, or run the harness against real arms. Claims marked "verified" were checked by running code on a scratch copy. All others come from reading the source.

Overall assessment

The question is worth asking, and the author deserves credit for preregistering a study of his own tools and for disclosing the conflict plainly. The design has real strengths: five archetypes, five payload sizes, two device profiles, five latency levels, a seeded data generator, a randomized build order, and nonparametric statistics suited to skewed timing data.

The package is not ready to register or freeze. Three kinds of problems stand in the way.

First, the code does not yet implement the study the preregistration describes. The control arms that carry the ablation (Arms 8 and 9) do not contain the patterns they are meant to test. Several archetypes omit the features that were supposed to create load (virtual scrolling, subtotal rows, the chart, the detail panel). The harness can run only a stub page, and its interaction scripts use selectors that do not exist in any arm. The two server-rendered htmx arms silently fail to delete or edit rows in the table archetype, so they do less work than the arms they are compared with.

Second, the measurement plan has confounds that favor the server-rendered arms or blur the comparison. Examples: INP does not include the network round trip of an htmx request; htmx loads from a public CDN without timing headers, so its bytes count as zero; Node and Python time the server differently; and the "0 ms" network profile also removes the bandwidth cap.

Third, the statistical plan defines error-control families too narrowly, gives no smallest effect of interest, tests "no difference" claims with tests that cannot support them, and adds data based on a variance rule that amounts to optional stopping.

Recommendation: major revision before registration. The fixes are concrete and most are mechanical. After they are made, the study could make a useful, practice-oriented contribution to SP&E. Until then, a registration would lock in a design whose results a skeptical reader could dismiss, and the conflict of interest makes that risk larger, not smaller.

Major issues

1. The ablation arms do not contain the patterns they are named for

Evidence. The preregistration says Arm 9 "uses all five conventional patterns (context providers, custom hooks, domain classes, conditional branching, component hierarchies)" (line 21). A search of all five Arm 9 folders found no `createContext`, no `Provider`, no custom `use*` hooks, no classes, no `switch` statements, and no error boundaries.

Arm 9 is close to Arm 7 with logic inlined. After whitespace is removed, Arms 7 and 9 differ by 44 to 97 lines per archetype. Because Arm 9 has no domain classes, Arm 8 (which has one) is more "conventional" than Arm 9, which inverts the intended order of the ablation.

Arm 8 does not use per-row domain objects. The task specification describes "an `Employee` class" with methods such as `fullName()` (task spec, line 43), and H11 rests on "per-object cost (domain classes, which scale with row count)." Each Arm 8 archetype instead wraps the whole dataset in a single model object (`TableModel`, `FormModel`, and so on). That is one instantiation per request, so there is no per-row cost to measure.

Arm 7, the Honest Code arm, breaks its own first rule. The table archetype's `App.jsx` uses eight `useState` calls plus `useCallback`, `useMemo`, and `useRef`. Its `format_cell` uses an `if` chain where the rules call for a configuration table.

The arms also do not do identical work. Arm 7 sorts strings with `localeCompare(..., { sensitivity: 'base' })`; Arm 9 uses plain `localeCompare`. Arm 9 sorts booleans in the reverse order. Arm 7 formats numbers with `'en-US'`; Arm 9 uses the default locale. Arm 8's getters re-run filter and sort on every call without memoization, so its client code differs from Arms 7 and 9. That contradicts the H10 claim that "all three hydrate to identical client-side code."

Fix. Rewrite Arm 9 so each of the five conventional patterns is present and identifiable, and list where each one appears. Give Arm 8 per-row domain objects as specified. Bring Arm 7 into line with the stated rules or restate the rules. Make the three arms produce identical HTML for identical input and add an automated test that compares their output byte for byte. Better still, have someone other than the author write Arm 9 (see Issue 6).

2. The arms do not implement the frozen task specification

Evidence.

- **Task specification out of date.** `task-specification.md` is labeled "Version 0.1 (draft) · Oct 25, 2026" and still describes nine arms with the old numbering ("SSR arms (6-8)," "React is the rendering engine across all nine arms," Arm 6 as "htmx + genX + domx"). It defines no Arm 10 or 11 and no React SSR plus stateless-domx arm. Lines 59 and 61 contradict each other on whether Arms 7 through 9 handle interactions on the client or the server.
- **Grid.** The specification calls for "No virtualization. All rows rendered," category subtotal rows, Tab navigation, shift-click cell ranges, a bulk-fill popover, and three-color variance. Every arm paginates at 25 rows, shows one totals row, uses a category filter and row checkboxes, and has two-color variance. Because only 25 rows render, payload size can no longer stress rendering, which is the stated purpose of this archetype.
- **Browser.** The specification calls for virtual scrolling with a shared utility (600 px viewport, 48 px rows). No arm has virtual scrolling, and `shared/` contains only `styles.css`. All arms paginate at 25 rows.
- **Dashboard.** No arm has a Chart.js chart (Chart.js is not even a dependency), an Apply button, a return-rate card, a detail panel, or 60-second auto-refresh. Filters apply on change.
- **Form.** The arms have no payment step and no Luhn check. Step 3 is account type. Form arms download the full payload (up to 100,000 records) and use only the first record, so "payload size" for the form archetype measures JSON download and parse time, not form behavior.

Fix. Freeze a task specification that matches the eleven-arm design, then build every arm to it. Add an acceptance test per archetype that checks each required feature through the DOM, and require all 55 combinations to pass before freeze. Either drop payload size as a factor for the form archetype or give it a real role.

3. The harness cannot yet run the study, and the interaction scripts do not match the apps

Evidence. `harness/src/runner.js` supports only `--validate`, which drives a stub server on port 3999 with `stubInteraction`. The full-run path prints "not yet implemented" (line 301). `metrics.js` throws on import.

I checked each selector in the five interaction scripts against all eleven arms:

Archetype	Selectors missing in all 11 arms
Table	`td[data-column="select"]` (checkbox cells); Arm 11 also lacks `data-column`, `filter-input`, `next-page`, and `delete-selected`
Grid	`data-column`, `bulk-fill-input`, `bulk-fill-apply`; rows are numbered from 1, but the script clicks `data-row="0"`
Dashboard	`startDate`, `endDate`, `apply-filters`, `<canvas>`, `orders-table`, `close-detail`
Browser	`virtual-list`, `batch-status`, `apply-batch`
Form	`[name=...]` inputs (arms use `data-field`), `sameAsBilling`, `cardholderName`, `data-action="next"`, `"back"`, `"submit"`

The smoke test (`smoke-nav.js`) admits this: interaction scripts "need data-attribute wiring that is not yet in every arm."

Other harness problems:

- The run order is sequential by condition, with no randomization or interleaving of arms, so thermal and background drift on the server are confounded with arm.
- One browser process is reused for all runs, while the preregistration promises fresh state per run.
- The summary printout takes min and max from an unsorted array, and the median function returns the upper value for even counts.

Fix. Finish the runner, make it fail loudly when any selector is missing, and add a check that each interaction actually changes the DOM as intended (for example, the row count drops after delete). Randomize or counterbalance the order of arms within each block, and record wall-clock start times so drift can be modeled. Run the full harness end to end on all 55 combinations before freeze and deposit that pilot log.

4. Measurement confounds favor the server-rendered arms

Evidence.

- **INP misses htmx network time.** INP measures from input to the next paint. An htmx click handler starts an asynchronous request and returns, so the browser paints before the server responds. The fragment swap happens later and is not part of that event's duration. H6 and H7 compare htmx INP with CSR INP, but the htmx number leaves out the round trip, which is the cost H7 is trying to find. The same problem affects debounced CSR filters, whose work runs in a `setTimeout` outside the event. As designed, H7's crossover probably cannot appear.
- **htmx comes from a public CDN.** Arms 10 and 11 load `https://unpkg.com/htmx.org@1.9.12`, which redirects (301) and then serves the file without a `Timing-Allow-Origin` header (verified with `curl`). Under the Resource Timing rules, `transferSize` for that file is reported as 0, so the collector undercounts the htmx arms' bytes. That biases H8, the claim that Arm 10 has the smallest bundle. The CDN fetch also adds an outside network dependency, a DNS lookup and TLS handshake for a second origin, and a redirect, none of which the CSR arms face.
- **No compression.** Neither server compresses responses, yet H3 and H8 speak of "gzipped" transfer size. No code computes gzipped size from build output, and parsed bundle size is not collected.

- **Server timing differs by runtime.** In `server.js` the timer starts before `readFileSync` and `JSON.parse` of the data file. In `fastapi-server.py` it starts after the data load. H12 compares those two numbers. For H10, Node's server time includes reading and parsing the full data file (up to 100,000 rows of JSON), not only rendering 25 rows, so the code-style difference could be buried in I/O.
- **Arm 11 has a different network path.** The Node server answers Arm 11 URLs with a 302 redirect to port 4001. Under 300 ms emulated latency, that adds a full round trip. The FastAPI server also does not send the COOP and COEP headers the Node server sends.
- **SSR arms ship the whole dataset.** Arms 6 through 9 inline the full payload as `window.__SSR_DATA__` in the HTML (up to tens of megabytes at 100,000 rows) while rendering 25 rows. That cost, not hydration or code style, could dominate their load metrics at large sizes. The htmx arms send only the visible page. This difference is part of each architecture and may be fair to measure, but the preregistration should say so and should not attribute it to state management or code style.
- **The "0 ms" profile removes the bandwidth cap.** In `profiles.js` the 0 ms profile sets throughput to unlimited, while the others cap it at 10 Mbps. The step from 0 to 20 ms changes bandwidth and latency together, which matters most for H6, H7, and H9.
- **TTI and heap.** The TTI routine has no network-quiet condition, so it is not the Lighthouse algorithm. It can also absorb long tasks from the interaction script, which begins two seconds after load. `peakHeap` is a single reading, not a peak, and `performance.memory` is quantized unless Chrome runs with precise memory info.
- **Bundling differs.** CSR and SSR client code is bundled and minified with esbuild. The htmx arms' scripts are served as shipped, and `HTMX_ARMS` in `build.js` is never built.

Fix. Add an interaction-latency metric that ends when the DOM reaches its final state (for example, a `MutationObserver` mark or a `data-ready` flag), and use it for H6 and H7. Keep INP as a secondary metric. Self-host every script, bundle every arm the same way, and turn on the same compression for both servers. Start and stop server timers at the same points in both runtimes, separate data-load time from render time, and load data once at startup. Serve Arm 11 on the same origin through a reverse proxy, or make every arm cross the same proxy. Cap bandwidth at 10 Mbps for the 0 ms profile too. Validate TTFB and TTI against a known-latency fixture before freeze.

5. The server-rendered arms do less work on the table archetype

Evidence (verified). Row IDs in the data are integers. The htmx arms pass them through query strings, where they become strings.

- In Arm 11, `delete_selected` checks `r["id"] not in selected` against string keys, so delete removes nothing (100 rows in, 100 rows out). `commit_edit` compares an integer with a string, so inline edits never apply.
- In Arm 10, `tableData.find(r => r.id === id)` compares a number with a string, so `/edit-cell` returns 404 and `/commit-edit` changes nothing. Delete works in Arm 10 because JavaScript coerces object keys.

Both servers also keep the dataset in a module-level variable that each page load resets, while the specification says, "The server does not maintain session state."

Fix. Fix the type handling, then add functional tests that assert the visible result of every scripted interaction for every arm. Treat "every interaction produces the specified end state" as a freeze criterion.

6. Conflict-of-interest controls are not yet auditable

Evidence.

- The research plan calls an independent programmer for the controls the strongest answer to the conflict (lines 50 to 54), but the preregistration does not adopt it. The author wrote every arm.
- Effort is visibly uneven. Arm 7 is the only arm with tests and BDD `.feature` files (15 feature files and 6 test files for the table archetype alone). Arm 9 is among the smallest implementations in every archetype.
- The tuning budget (eight hours to build, four to tune, per arm) cannot be audited. There is no time log, the commit history shows no build order, and all 55 combinations arrived in one commit along with the "randomized" implementation order. The order file reproduces from its seed, but it lists all eleven arm names, so it was generated after the eleven-arm design and cannot show that work followed it.
- A `CLAUDE.md` file suggests an AI coding assistant helped write the code. If so, hours are a weak measure of effort, and the prompts matter as much as the time.
- The freeze has no timestamp or Zenodo DOI yet. The conflict statement names Arms 6, 7, and 11 (line 272), but the description names Arms 6 and 7 (line 23).

Fix. Recruit at least one independent React developer to write or review Arms 1 through 4, 8, and 9 against the corrected specification, and publish their sign-off. Keep a per-arm time log or commit-by-commit history, or replace the hour budget with a review-based parity criterion. If an AI assistant was used, disclose it, and archive the prompts or apply the same prompting procedure to every arm. Mint the Zenodo DOI and freeze tag before any data collection, and state in the registration that no pilot data on real arms have been seen, or report what has.

7. The statistical plan controls error in families that are too small, and it tests null predictions incorrectly

Evidence.

- **Families.** H1 through H4 apply Bonferroni across four comparisons "for each metric, at each payload size." The factors also include five archetypes, two devices, and five latencies. If H1 is tested in every cell, that is $4 \times 5 \times 5 \times 2 \times 5 = 1{,}000$ Mann-Whitney tests for INP alone, each at $\alpha = .0125$. Across the roughly dozen listed metrics, the count passes 10,000. "At every payload size" makes H1 a conjunction of many tests, but no rule says how they combine into one decision.
- **Power.** The power statement (line 121) asserts adequacy without a calculation. With 30 runs per group, a Mann-Whitney test at $\alpha = .0125$ has about 80 percent power only for large effects (a Cliff's delta of roughly 0.45 under normal approximations). Browser timings often differ by more than that, so low power is not the main risk. The larger risk is the opposite: with this many tests, trivially small differences in milliseconds will reach significance. The plan names no smallest effect of practical interest.
- **Null predictions.** H10 (client metrics) and H12 predict "no difference" and test it with a nonsignificant result. Nonsignificance does not show equivalence. H12's "within 20 percent" has no test at all.
- **H4.** H4 predicts an interaction (device by arm) but is judged by eyeballing which Cliff's delta is larger. No test is specified.
- **H1's trend.** "The difference will increase as payload size increases" has no test.
- **Optional stopping.** Line 123 adds 30 more runs when CV exceeds 0.30. Adding data after a look inflates error unless it is planned for, and high-variance conditions may differ by arm.
- **Independence.** Runs within a condition are consecutive, so they share drift. Archetypes and payload sizes come from the same data generator and server process. Kruskal-Wallis and Mann-Whitney assume independent observations.
- **Metric inconsistencies.** Server response time is listed under "Recorded but not analyzed" (lines 193 to 202) but is the main outcome of confirmatory H10 and H12. ScriptDuration (H13) and CLS are listed both as outcomes and as "not analyzed." The CSV writer has no Server-Timing column, so H10 and H12 have no data path. H6 and H7 refer to "the htmx+genX+domx arm," an old name. Line 266 says "75,000+ runs" while line 117 says 82,500.

Fix. For each confirmatory hypothesis, name one primary metric, the exact cells it is tested in, and the rule that turns many cell results into one decision. Control error across that whole family (Holm is uniformly better than Bonferroni), or fit a model over the factors. For timing data, a mixed or quantile regression on log time with arm, payload, device, latency, and their interactions, and with a run-order term, answers H1, H4, and the trend claim in one place. State a smallest effect of interest in milliseconds or as a percent for each metric. Test the "no difference" claims with an equivalence procedure such as TOST, using those bounds. Replace the CV rule with a fixed run count, or preregister a sequential design with alpha spending. Decide how outliers are handled before collection. Make the metric table, the analysis plan, and the CSV columns agree.

8. Data generation is not reproducible across time zones

Evidence (verified). `generate-data.js` builds dates with `new Date(startYear, 0, 1)`, which uses local time, and then calls `toISOString()`. Running the generator under `TZ=America/Chicago` reproduces the checked-in files exactly. Under `UTC` and `Asia/Tokyo`, all five sizes of the table, dashboard, and browser data differ. In `table/1000.json`, 245 of 1,000 rows change, for example `hireDate` 2016-06-10 versus 2016-06-11. Form and grid data are unaffected. There is also no `package-lock.json`. Dependencies use caret ranges, and the npm versions of `stateless-dataos` and `domx-dataos` may not match the minified copies in `vendor/` that Arms 10 and 11 serve.

Fix. Build dates in UTC (`Date.UTC`), commit a lockfile, pin exact versions, and record SHA-256 hashes of every data file and vendored script in the freeze deposit.

Minor issues

1. H3 and H8 say "at every payload size," but bundle size does not depend on payload. H8 also calls bundle size a single measurement, which leaves nothing to test.
2. H6 cites Arm 10, while the analysis section says "htmx+genX+domx arm." Use one name.
3. The preregistration says Arm 6 isolates "the DATAOS pattern as the single variable against Arm 9." Arm 6 also differs from Arm 9 in server code style. Compare it with Arm 7.
4. H12 says Arms 10 and 11 "deliver the same HTML fragments." They use different templates, and Arm 11 also loads domx and genX.
5. Blinding is "not applicable" (line 79). The analysis can be blinded by recoding arm labels until the analysis script is frozen. Deposit the analysis script before collection; none is in the repository.
6. The TTFB note (line 183) asserts how CDP latency emulation interacts with `requestStart`. Validate that claim with a fixture.
7. INP is reported as the 98th percentile across interactions within a run. With about 10 scripted interactions, that is the maximum. Say so, and report per-interaction latencies as well.
8. `server.js` interpolates the `rows` query value without validation. This is harmless locally but should be checked against the five allowed sizes.
9. EJS renders templates without caching, while Jinja2 caches them. Turn on EJS caching for parity.
10. The FastAPI server runs a single uvicorn worker, and Express runs a single process. State this, and state whether the server shares CPUs with the browser.
11. `data/` is 128 MB in git. Consider generating it at build time and depositing hashes.
12. The VPS description (8 vCPU, Haswell) should include the provider, the instance type, the kernel, the Chrome version, and whether CPU frequency scaling is fixed.
13. The mobile profile throttles CPU only. Say that it does not emulate mobile memory, GPU, or screen size.
14. Add a threats-to-validity section that addresses construct, internal, external, and conclusion validity, as SP&E reviewers will expect.

Review by area

1. Experimental design and confounds

The factorial structure is sound in outline. The central weakness is that the arms differ in more than the named variable. The CSR arms vary only the state library, which is the cleanest comparison in the study, provided the five apps do identical work. The server-rendered arms differ in data shipped, network path, origin, bundling, script source, server runtime, and timer placement (Issue 4). The ablation does not vary what it claims to vary (Issue 1). The grid and browser archetypes, as built, no longer create the load they were chosen to create (Issue 2). Run order is not randomized, so time drift is confounded with arm (Issue 3).

2. Statistical plan

Nonparametric tests and Cliff's delta suit skewed timing data. The rest needs work: family definitions, a smallest effect of interest, equivalence tests for null predictions, a real test for H4 and the H1 trend, a fixed run count, and a model that respects the factor structure and run order (Issue 7). Confidence intervals for Cliff's delta should name the method used to compute them.

3. Falsifiability of hypotheses

H1, H2, H6, and H9 make directional predictions that data could contradict, which is good. Several others are weak:

- H5, H7, H11, and H13 are exploratory and are correctly labeled.
- H4 has no decision rule.
- H10's interpretation is unfalsifiable as written. If Arm 8 lands closer to Arm 7, tree depth is "the primary driver"; if closer to Arm 9, domain classes are. Either outcome confirms some story. Add a third outcome ("no meaningful difference among 7, 8, and 9") with an equivalence bound.
- H9 says, "We predict all three SSR arms win at 50ms+ and lose at 0-20ms; we report whichever direction the data shows." State in advance which outcome counts as disconfirmation.
- H12's "within 20 percent" needs a test and a bound.

4. Scope and limitations

The scope paragraph (line 25) is candid about which application types the results do not cover, and that is to the author's credit. Missing limitations:

- Results come from synthetic data on one machine, one browser engine, and React 18 only.
- The server-rendered arms keep state in query strings, which works only for small state.
- Payload size means rows shipped for CSR and SSR arms but rows on the server for htmx arms.
- The author built every arm.

The research plan's claim that the "zero sync bugs" property is analytic and need not be measured (line 68) should be kept out of the paper's claims or stated as a hypothesis the study does not test.

5. Conflict-of-interest management

Disclosure is clear. Management is weaker than the documents suggest (Issue 6). The research plan itself identifies independent implementation as the strongest remedy, and the preregistration does not adopt it. The tuning budget is symmetric on paper but unverifiable in practice, and visible effort is not symmetric. The freeze has no timestamp. The single initial commit means a reader cannot confirm that the specification and build order came before the code.

6. Code-to-registration alignment

- **All 55 combinations exist.** Each of the five archetype folders holds eleven arm folders.
- **Shared components.** Shared CSS exists and is linked from every page template I checked. The shared virtual-scroll utility and Chart.js do not exist. The shared Express server exists. The FastAPI server is separate and times differently.
- **Independent-variable isolation.** It fails for the ablation (Issue 1) and the server comparisons (Issue 4).
- **Metric coverage.** The preregistration's metric table against the collector:

Preregistered metric	Collected?	Note
Gzipped bundle size	No	No compression on the server, and no build-output measure. CDN script counts as 0
Parsed bundle size	No	
TTFB	Yes	Definition needs validation
FCP, LCP, CLS	Yes	
TTI	Yes	Not the Lighthouse algorithm (no network-quiet rule)
INP (p98)	Yes	Excludes htmx round trip. p98 of about 10 interactions is the max
TBT	Yes	Depends on the TTI definition
JS heap	Partly	Single reading, not peak
Script, Layout, RecalcStyle, Task duration	Yes	Cumulative over load plus interactions plus idle
Server response time (Server-Timing)	Read in browser	Not written to the CSV. Timed differently in Node and Python
Re-render counts, fragment size, lines of code	No	Mentioned in planning documents, not collected

- **Interaction scripts against the specification.** The scripts follow the preregistered sequences, but the apps do not implement those sequences, and the selectors do not match (Issue 3).
- **Build configuration.** CSR and SSR client bundles use esbuild, minified ES modules targeting Chrome 120 in production mode. The htmx arms are not bundled, and there is no lockfile.

7. Fit for *Software: Practice and Experience*

The topic fits. SP&E publishes empirical comparisons of practical engineering choices, and practitioners want this question answered. To meet SP&E's expectations, the paper should frame itself as a controlled benchmark of specific implementations, not of architectures in general. It should give a threats-to-validity section, archive code and data, and keep claims tied to measured outcomes. The research plan's framing as answering "one objection" and as a test of a book's discipline will read as advocacy to SP&E reviewers. I did not confirm whether SP&E offers a registered-report track. If it does not, the OSF registration still adds credibility, but the paper will be judged on its results and methods like any other submission.

8. Advocacy language

The preregistration is mostly neutral. Most advocacy is in the research plan and the task specification, which will be deposited with the registration and may be quoted in the paper.

Location	Current wording	Suggested neutral wording
Research plan, line 5	"Every design choice below answers one objection: that the architecture was benchmarked by its own	"The design aims to limit bias from the author's role as developer of two of the approaches under test."

Location	Current wording	Suggested neutral wording
	author."	
Research plan, line 60	"Preregistration is the only instrument that answers that objection, and it answers it completely."	"Preregistration reduces this risk by fixing hypotheses and analyses before data collection. It does not remove the risk that the author's implementations favor his own approach."
Research plan, line 68	"Measuring it would be a category error, and a reviewer would say so."	"This study does not measure synchronization defects. We argue in a separate section that a single copy of state rules out one class of them."
Research plan, line 74	"That is a stronger and more honest result than any benchmark"	"This argument complements the benchmark; it does not replace it."
Research plan, line 82	"says something the field does not currently have."	"To our knowledge, no published study measures hydration cost at three levels of client-side state with rendering mode held fixed."
Task spec, line 23	"This matches the architecture multicardz uses in production."	Delete, or move it to the conflict-of-interest statement.
Task spec, line 137	"every major grid library (AG Grid, Handsontable, react-datasheet-grid), all of which bypass React's rendering model to achieve acceptable performance."	"Several grid libraries, including AG Grid and Handsontable, manage cell rendering outside React's normal update path [cite]."
Task spec, line 156	"In conventional React, this cascades ... a single keystroke touches roughly 2,000 components"	"In an implementation that passes grid state through context or props, an edit can trigger re-renders of many dependent components. We measure the actual count."
Task spec, line 158	"The cost scales with the number of changed cells, not with the total number of cells in the grid."	"We expect server cost to scale with changed cells rather than total cells; H11 tests this."
Task spec, line 234	"The controlled-input problem does not exist because the input was never controlled."	"Arm 6 uses uncontrolled inputs, so keystrokes do not trigger component updates. The form archetype measures whether this changes INP."
Preregistration, line 21	"Arms 7, 8, and 9 form a three-point ablation that identifies which patterns drive any performance gap."	"Arms 7, 8, and 9 form a three-point ablation intended to show which patterns contribute to any performance gap."
Preregistration, line 23	"applying them to mainstream web development, where they remain the minority pattern."	Delete, or cite evidence for "minority pattern."
Preregistration, lines 49 and 53	"because all three hydrate to identical client-side code"; "both deliver the same HTML fragments"	State these as design requirements verified by tests, not as facts, and remove "because."
Repository name and arm labels	"Honest Code" as a label for one arm, against "conventional" for others	Use descriptive labels in the paper, such as "functional-style server components," and mention the book name once in the disclosure.

What I did not verify

- I did not install dependencies, build the apps, start either server, or run the smoke test, so I have not confirmed that every page loads.
- I did not measure bundle sizes, server times, or any browser metric.
- The INP and Resource Timing points rest on documented browser behavior and on reading the code, not on a test run.
- I did not check the local copy on the author's machine, so files that exist only there are not reflected here.
- I did not confirm SP&E's current article types.